PROJETO ESG + DEVOPS

Cidades ESG Inteligentes

Órbita Alerta

Documentação técnica da entrega — Navegando pelo mundo DevOps

Integrante	Micaela Guimarães — FIAP
Repositório	github.com/mguimaraesfiap/orbita-alerta-devops
Staging	https://orbita-alerta-staging-mg.azurewebsites.net
Produção	https://orbita-alerta-production-mg.azurewebsites.net
Stack	C# / .NET 8, MongoDB, Docker, Docker Compose, GitHub Actions, GHCR, Azure App Service e Azure Cosmos DB (API MongoDB)

Documento gerado em 13/10/2026

1. Contexto e objetivo

O Órbita Alerta é uma API ESG para registro de alertas ambientais de cidades inteligentes. Nesta fase, o projeto C# foi adaptado para simular um ambiente de produção: build e testes automatizados, imagem Docker, orquestração com Docker Compose e deploy automatizado em dois ambientes na nuvem (staging e produção).

Endpoints: GET /health (estado da aplicação e ambiente), GET /api/alerts (lista alertas ordenados por data) e POST /api/alerts (cria alerta com validação de título, categoria, severidade e cidade).

2. Pipeline CI/CD

Ferramenta: GitHub Actions, arquivo .github/workflows/ci-cd.yml. O workflow é acionado em pull requests e pushes nas branches develop, qa e main. Os jobs são encadeados por needs: nada é publicado se o build ou os testes falharem, e o mesmo artefato (imagem identificada pelo SHA do commit) é promovido para os ambientes.

Job	Quando executa	O que faz
Build e testes	PR e push (develop, qa, main)	Setup do .NET 8, restore, build Release, testes xUnit com cobertura e upload dos resultados (.trx) como artefato.
Build e publicação da imagem	push em qa ou main	Login no GHCR com GITHUB_TOKEN, docker build multi-stage e push com as tags <sha> e <branch>-latest.
Deploy staging	push em qa	azure/webapps-deploy atualiza o Web App de staging com a imagem do commit; em seguida, smoke test em /health e /api/alerts.
Deploy production	push em main	Mesmo processo no Web App de produção (GitHub Environment production), com smoke test ao final.

Fluxo de promoção

```
feature -> develop (build + testes)
        -> qa      (build + testes -> imagem GHCR -> deploy staging    -> smoke test)
        -> main    (build + testes -> imagem GHCR -> deploy production -> smoke test)
```

Segredos usados pelo pipeline (GitHub → Settings → Secrets): AZURE_WEBAPP_STAGING, AZURE_WEBAPP_STAGING_PUBLISH_PROFILE, AZURE_WEBAPP_PRODUCTION e AZURE_WEBAPP_PRODUCTION_PUBLISH_PROFILE. A string de conexão do banco fica somente nas configurações de cada Web App (MONGO_CONNECTION_STRING e MONGO_DATABASE), fora do repositório.

3. Docker: arquitetura, comandos e imagem

O Dockerfile usa build multi-stage: a imagem do SDK compila e publica; a imagem final contém apenas o runtime ASP.NET Core e os binários publicados, roda na porta 8080 e com o usuário não privilegiado app.

```
# syntax=docker/dockerfile:1

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["src/OrbitaAlerta.Api/OrbitaAlerta.Api.csproj", "src/OrbitaAlerta.Api/"]
RUN dotnet restore "src/OrbitaAlerta.Api/OrbitaAlerta.Api.csproj"
COPY . .
RUN dotnet publish "src/OrbitaAlerta.Api/OrbitaAlerta.Api.csproj" \
    --configuration Release \
    --output /app/publish \
    --no-restore \
    /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
EXPOSE 8080
```

```
ENV ASPNETCORE_URLS=http://+:8080
ENV DOTNET_SYSTEM_GLOBALIZATION_INVARIANT=false
COPY --from=build /app/publish .
USER app
ENTRYPOINT ["dotnet", "OrbitaAlerta.Api.dll"]
```

Orquestração com Docker Compose

• api: build do Dockerfile, imagem orbita-alerta-api:local, porta ${API_PORT:-8080}, variáveis ASPNETCORE_ENVIRONMENT, MONGO_CONNECTION_STRING e MONGO_DATABASE, sobe somente após o MongoDB ficar saudável (depends_on + healthcheck).

• mongodb: mongo:7.0 com usuário root definido por variáveis do .env e healthcheck via mongosh.

• Volumes nomeados orbita-alerta-mongodb-data e orbita-alerta-mongodb-config preservam os dados entre reinícios.

• Rede bridge orbita-alerta-network isola a comunicação entre API e banco.

• .env.example documenta as variáveis; o .env real não é versionado nem copiado para a imagem (.dockerignore).

Comandos utilizados

```
Copy-Item .env.example .env
docker compose up --build -d
docker compose ps
Invoke-RestMethod http://localhost:8080/health
docker compose logs -f api
docker compose down        # mantém o volume do MongoDB
```

Ambientes em nuvem (Azure for Students, capacidade mínima)

Recurso	Configuração
App Service Plan plan-orbita-alerta	Linux, SKU F1 (gratuito), North Central US
Web App orbita-alerta-staging-mg	Container da imagem GHCR, ASPNETCORE_ENVIRONMENT=Staging, banco orbita_alerta_staging
Web App orbita-alerta-production-mg	Container da imagem GHCR, ASPNETCORE_ENVIRONMENT=Production, banco orbita_alerta_production
Cosmos DB orbita-alerta-cosmos-mg	API MongoDB, free tier (1000 RU/s gratuitos), 400 RU/s por banco, South Africa North

4. Evidências: pipeline (build, testes e deploy)

Os prints de terminal são capturas reais de janelas do Windows PowerShell executando os comandos no computador da integrante (script scripts/collect-evidence.ps1); as saídas em texto ficam em docs/evidencias/terminal. Os prints do GitHub foram capturados na interface web do repositório.

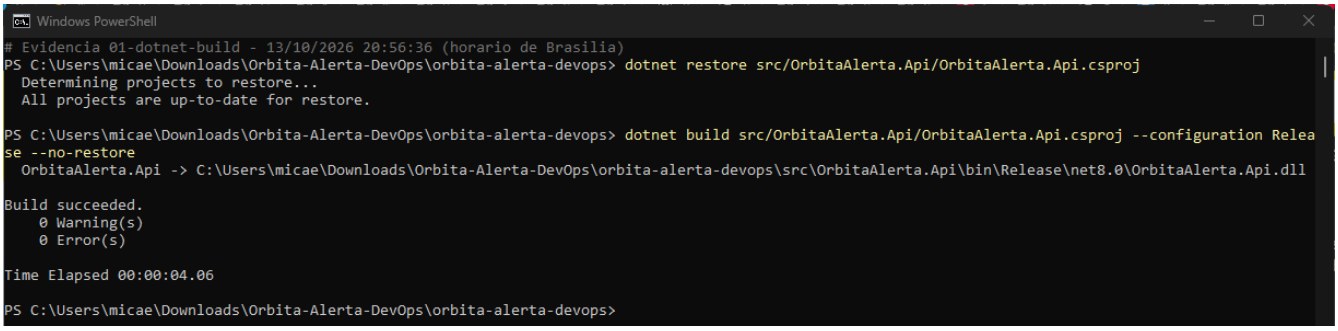


Figura 1 — Restore e build Release da API.

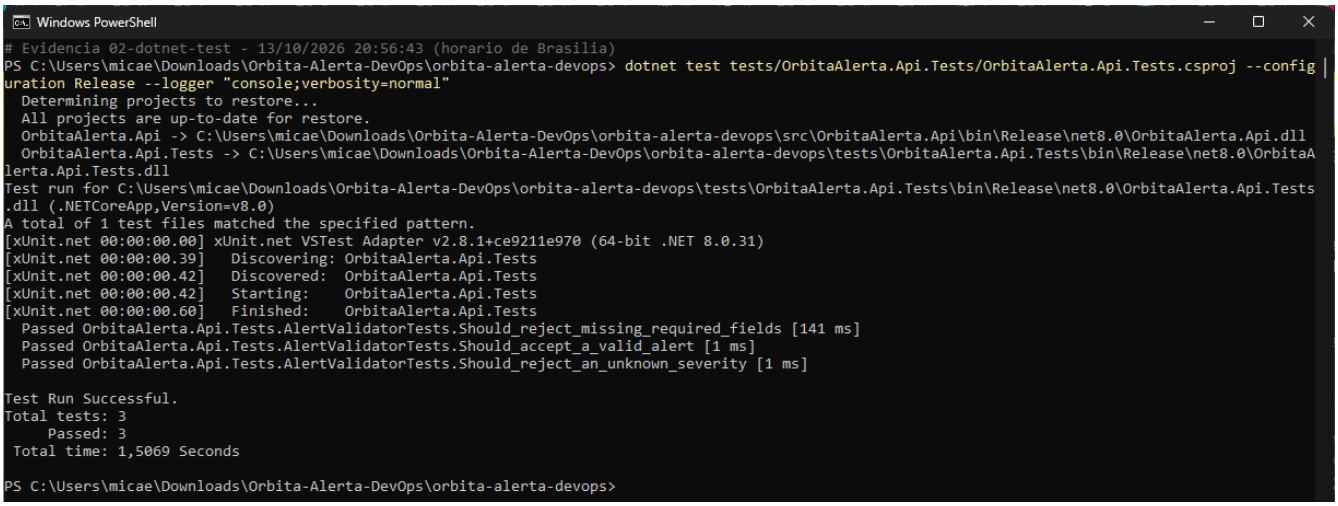


Figura 2 — Testes automatizados xUnit: 3 de 3 aprovados.

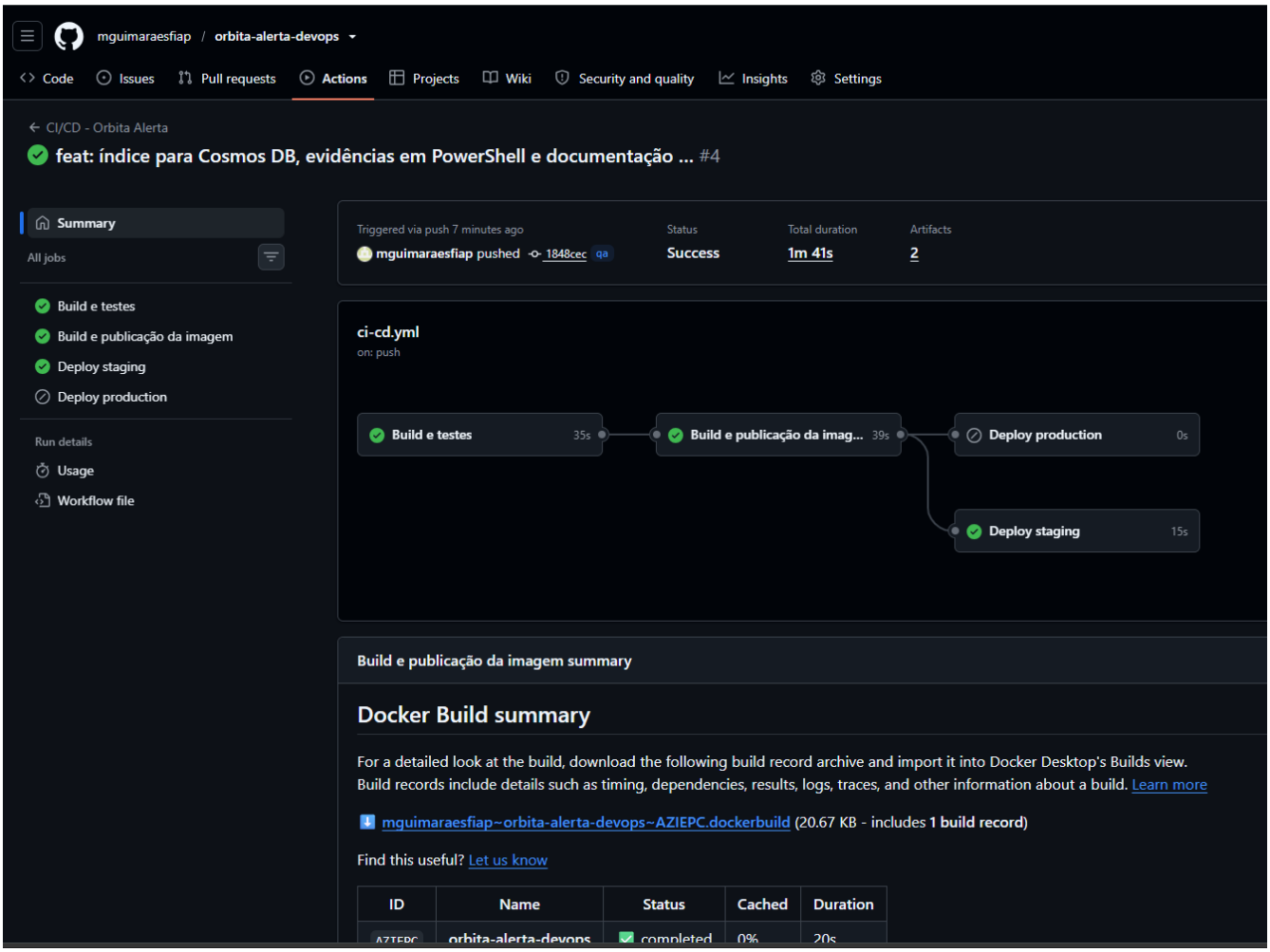


Figura 3 — GitHub Actions, run #4 (branch qa): build e testes, publicação da imagem e Deploy staging concluídos com sucesso.

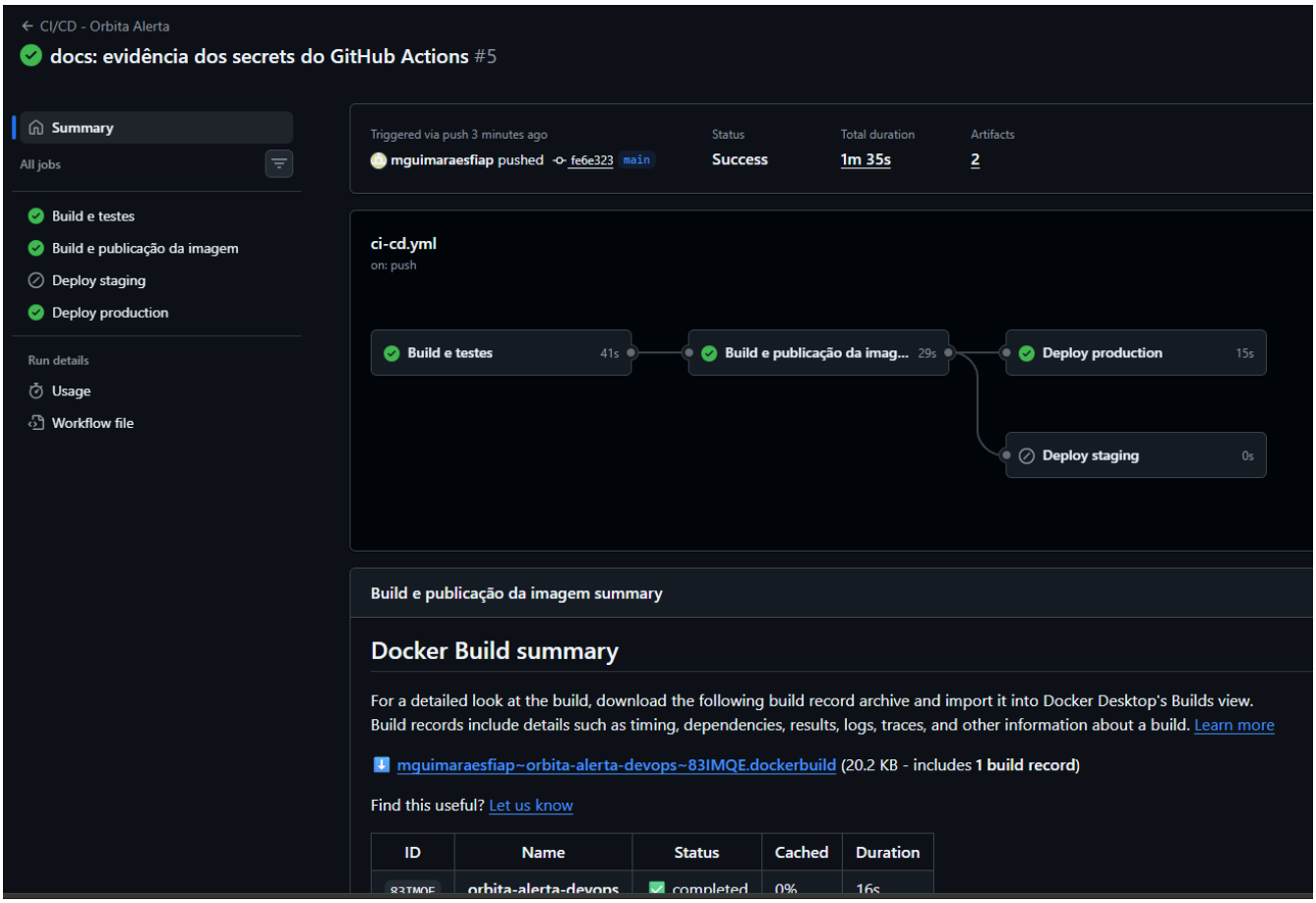


Figura 4 — GitHub Actions, run #5 (branch main): build e testes, publicação da imagem e Deploy production concluídos com sucesso.

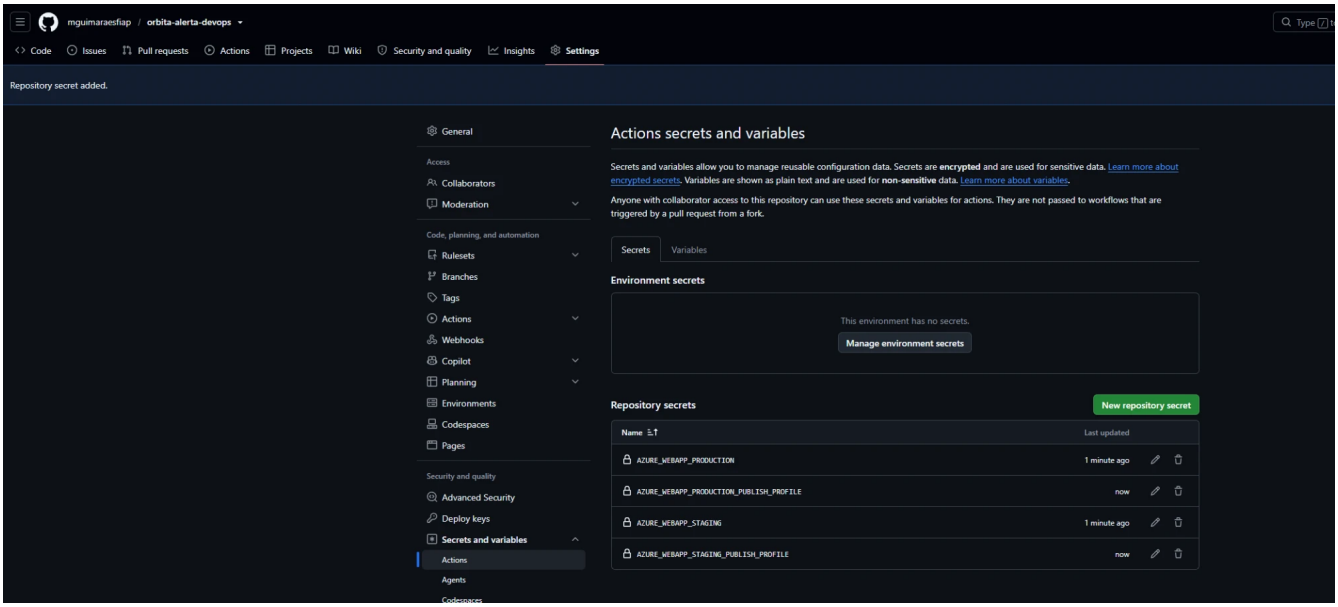


Figura 5 — Secrets do Azure cadastrados no repositório GitHub (valores criptografados, não exibidos).

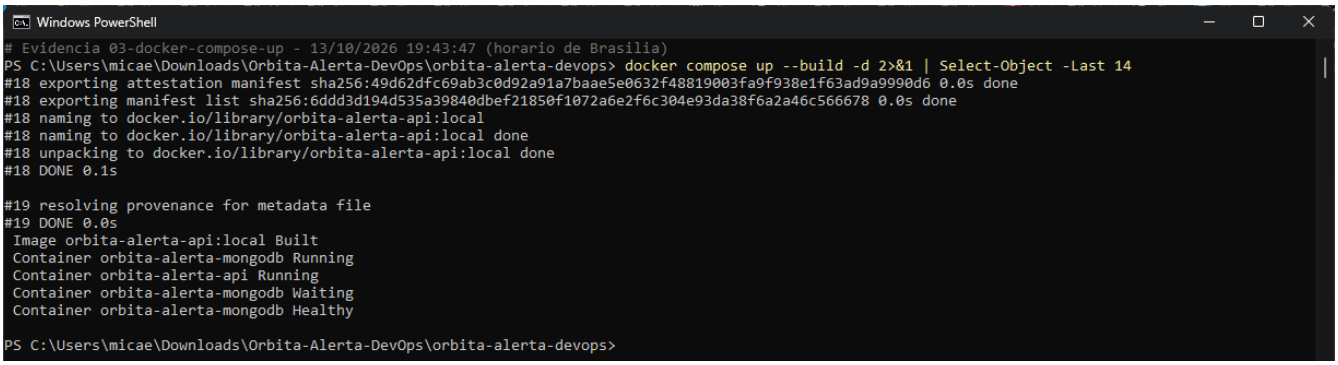


Figura 6 — Build da imagem multi-stage e subida dos containers com Docker Compose.

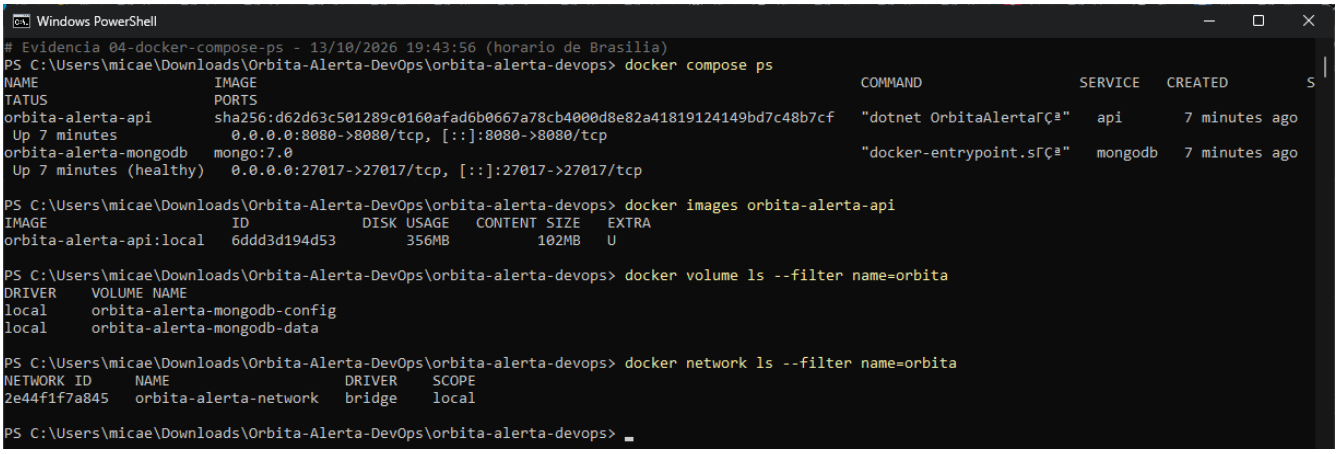


Figura 7 — Containers em execução, imagem criada, volumes e rede.

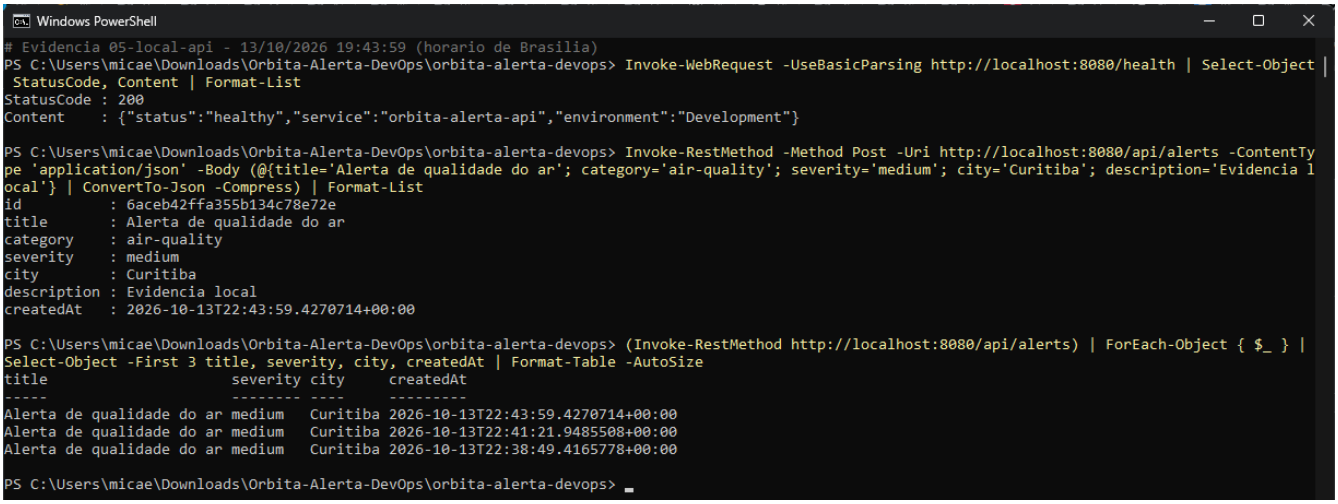


Figura 8 — API local: healthcheck, criação e listagem de alertas.

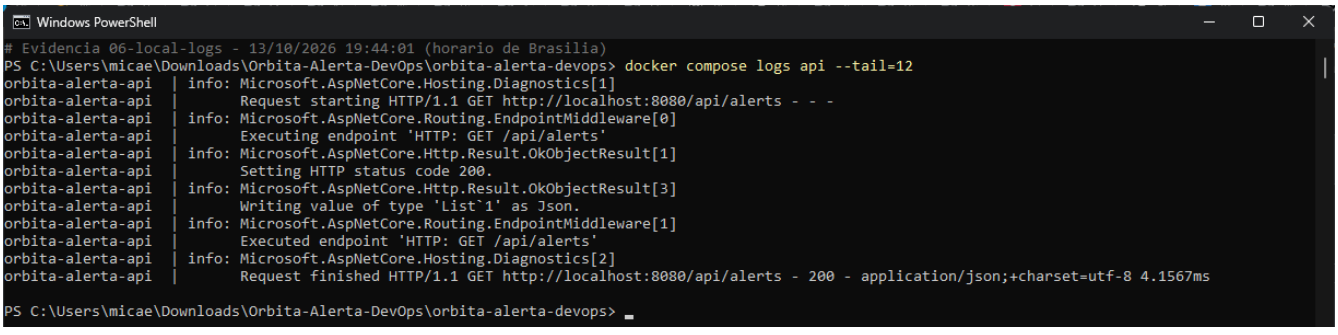


Figura 9 — Logs do container da API.

5. Evidências: ambientes staging e produção

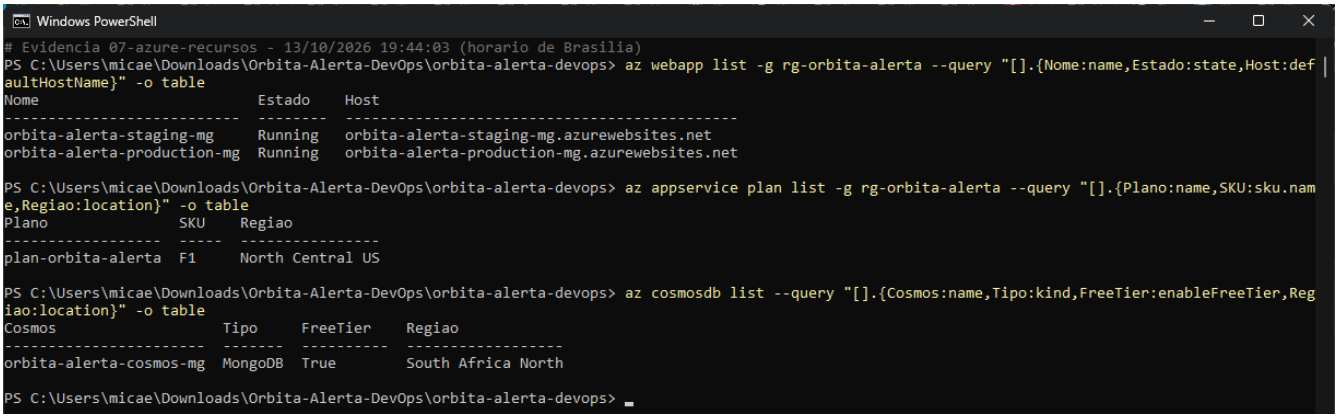


Figura 10 — Recursos Azure: Web Apps de staging e produção em execução, plano F1 e Cosmos DB free tier.

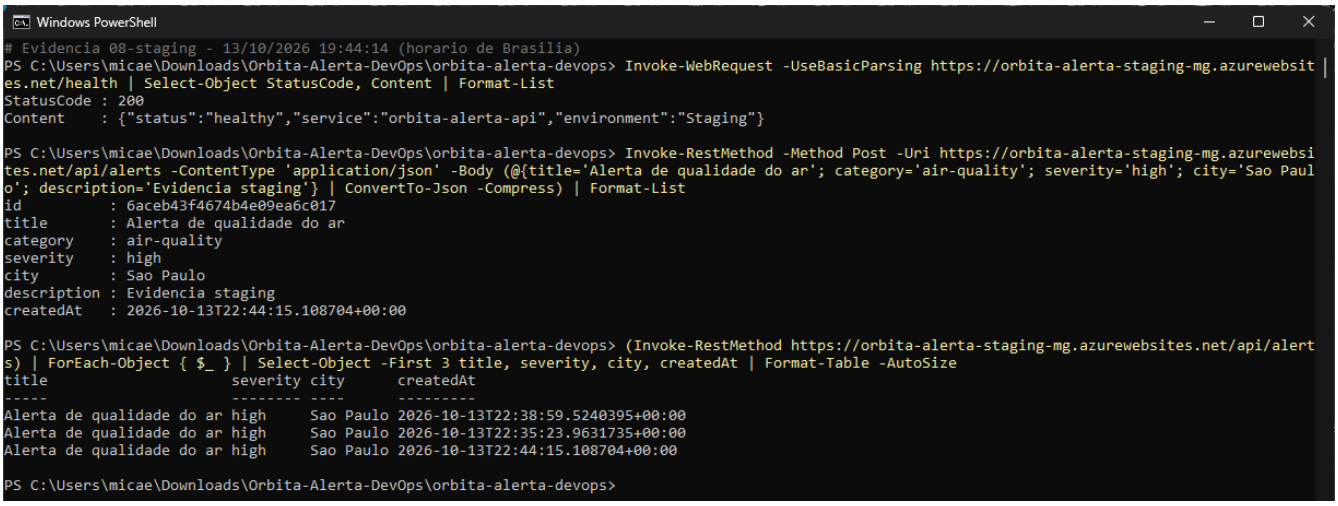


Figura 11 — Staging (orbita-alerta-staging-mg.azurewebsites.net): /health com environment=Staging, POST e GET de alertas.

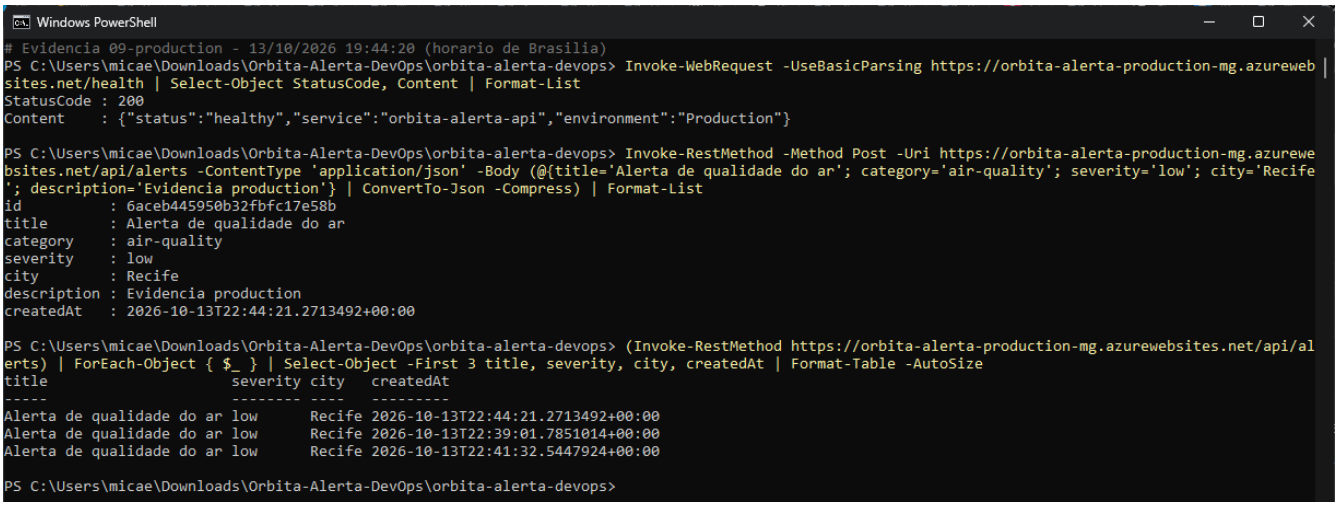


Figura 12 — Produção (orbita-alerta-production-mg.azurewebsites.net): /health com environment=Production, POST e GET de alertas.

6. Desafios encontrados e soluções

Desafio	Solução
Primeiro deploy de produção falhou com "No credentials found": o workflow dependia de secrets ainda não cadastrados e a branch qa não existia.	Criação dos Web Apps, habilitação das credenciais de publicação (basic auth do SCM), cadastro dos publish profiles como secrets e criação da branch qa para o fluxo de staging.
A assinatura Azure for Students só permite 5 regiões; o Cosmos DB herdava a região do resource group (eastus) e era bloqueado pela política.	Resource group dedicado ao banco, criado em uma região permitida.
North Central US sem capacidade para novas contas Cosmos DB no momento da criação.	Banco criado em South Africa North; a API acessa o banco pela string de conexão, independente da região.
O MongoDB local (Docker Desktop) não é acessível pelos ambientes em nuvem.	Cosmos DB com API MongoDB no free tier, um banco por ambiente (orbita_alerta_staging e orbita_alerta_production).
GET /api/alerts retornava HTTP 500 no Azure: o Cosmos DB só ordena por campos indexados (no MongoDB local o índice é opcional).	A aplicação passou a criar o índice CreatedAt na inicialização (MongoAlertRepository) e o índice foi criado nas collections existentes.
Manter custo zero e não expor credenciais.	Plano F1 gratuito, Cosmos free tier, segredos somente em GitHub Secrets e nas configurações do App Service; .env e publish profiles no .gitignore.

7. Checklist de entrega

Item	OK	Observação
Projeto compactado em .ZIP com estrutura organizada	☑	Dockerfile, docker-compose.yml, src/, tests/, scripts/, docs/, README.md, .env.example e .github/workflows/.
Dockerfile funcional	☑	Multi-stage .NET 8, usuário não privilegiado (Figura 6).
docker-compose.yml ou arquivos Kubernetes	☑	API + MongoDB com volumes, variáveis de ambiente, rede e healthcheck (Figura 7).
Pipeline com etapas de build, teste e deploy	☑	GitHub Actions: build, testes, imagem GHCR e deploy (Figuras 3 a 5).
README.md com instruções e prints	☑	Execução local, pipeline, containerização, prints e tecnologias.
Documentação técnica com evidências (PDF ou PPT)	☑	Este PDF.
Deploy realizado nos ambientes staging e produção	☑	Deploy pelo pipeline concluído em staging (run #4) e produção (run #5), com smoke test (Figuras 3, 4, 11 e 12).